Resumen de la sesión — viernes 25-09-2026

Proyecto **Sabores de Colombia** · carpeta `clases/arrays/` · reconstruido a partir del commit `024a138` y de la bitácora de reflexiones

Qué se construyó

#	Qué	Dónde
1	Arreglo de `EliminarElemento` (la función estaba vacía)	`array1.html`
2	Separación de datos y lógica	`proyectos/data.js` + `script.js`
3	Modal de detalle al hacer clic en una tarjeta	`proyectos/index.html` + `script.js`
4	CRUD completo de platos	`proyectos/gestion.html` + `gestion.js`
5	Persistencia con `localStorage`	`proyectos/data.js`
6	Login que oculta el CRUD	`CREDENCIALES.JS` + `gestion.js`

1. EliminarElemento con splice

Qué pasó: el botón existía, pero la función estaba vacía y no tenía ningún listener (función que "escucha" un evento, como un clic, y reacciona). Ahora busca la marca con `indexOf` y la saca con `splice(pos, 1)`.

Por qué splice y no pop o shift

- `pop()` solo saca el **último** elemento y `shift()` solo el **primero**. Trabajan en los extremos.
- `splice(posición, cantidad)` es el único de los tres que recibe una posición cualquiera, así que es el que sirve para borrar uno específico.
- El patrón "primero ubico con `indexOf`, después actúo" se repite en todo el proyecto. `indexOf` devuelve `-1` si no encuentra nada, y por eso se revisa ese caso antes de borrar.

Concepto que se refuerza: `indexOf` compara el texto de forma estricta, así que `"Nike"` y `"nike"` son valores distintos. Por eso se usa `.toLowerCase()`. Es lo mismo que corregimos el 21-09 en `buscarElemento`.

2. Separar data.js de script.js

Qué pasó: el array `platosColombia` (array de objetos: cada plato es un objeto con las mismas 6 propiedades) pasó a su propio archivo.

Por qué

- **Una sola fuente de datos para dos páginas.** Tanto `index.html` como `gestion.html` cargan `data.js`. Si los datos estuvieran copiados en cada página, cambiarlos en una no cambiaría la otra.
- **El orden de los `<script>` importa.** `data.js` se carga antes que `script.js` y `gestion.js` porque esos archivos usan `platosColombia`. Si el orden se invierte, la variable todavía no existe y aparece un `ReferenceError`.

- **Paralelo con la producción audiovisual:** es como tener la ficha de personaje (vestuario, paleta) en un documento aparte que consultan todas las escenas, en vez de reescribirla en cada prompt. Si se cambia en un lugar, la continuidad se mantiene.

3. Tarjetas con <template> y modal de detalle

Cómo se arman las tarjetas: `<template>` es un bloque de HTML que el navegador guarda pero no muestra. `script.js` lo clona una vez por plato (`cloneNode(true)`) y llena los huecos `.js-*`. Ninguna tarjeta está escrita a mano en el HTML.

El modal (ventana que aparece encima de la página y oscurece el fondo):

- Hay **un solo modal** para todas las tarjetas. `abrirModal(plato)` lo rellena con los datos del plato en el que se hizo clic y le quita el atributo `hidden`.
- Se cierra de tres formas: con el botón de cerrar, con un clic en el fondo oscuro (`evento.target === modal`) o con la tecla Escape (convención estándar de accesibilidad).

El concepto más importante de esta parte: el closure

Un closure es una función que "recuerda" las variables del lugar donde se creó.

```
carta.querySelector(".food-card")
     .addEventListener("click", () => abrirModal(plato));
```

El clic ocurre **mucho después** de que terminó el `forEach`. Aun así, cada tarjeta abre su propio plato y no el último del array. Eso pasa porque la función flecha se creó dentro de esa vuelta del bucle y guarda una referencia a ese `plato`. Vas a volver a ver esta idea cada vez que agregues listeners dentro de un bucle.

4. CRUD en gestion.html

CRUD son las cuatro operaciones básicas sobre datos: **C**reate (crear), **R**ead (leer), **U**pdate (actualizar), **D**elete (borrar).

Operación	Cómo lo hace	Método clave
Read	Recorre el array y arma una fila `<tr>` por plato	`forEach` + `createElement`
Create	Lee el formulario, arma un objeto con la misma forma que los de `data.js` y lo agrega	`FormData` + `push`
Update	Un `<select>` guarda el índice de cada plato; al enviar, reemplaza el objeto completo en esa posición	`platosColombia[indice] = {...}`
Delete	Botón por fila con `data-index`; pide confirmación y lo saca del array	`splice(indice, 1)`

Conceptos detrás de cada decisión

- `evento.preventDefault()`: por defecto, un `<form>` recarga la página al enviarse. Sin esta línea, la página se recargaría y se perdería lo que el JS acababa de hacer.
- `FormData` + atributo `name`: `FormData` lee los campos por su `name`. Por eso los inputs se llaman igual que las propiedades del objeto (`nombre`, `region`...) y armar el objeto queda directo.
- **Convertir tipos:** todo lo que llega de un formulario es texto. `Number(...)` convierte el tiempo en número; en el checkbox, `=== "on"` lo convierte en booleano; los sabores pasan de `"Dulce, Salado"` a `["Dulce", "Salado"]` con `.split(",").map(trim).filter(no`

`vacío)`.

- **Delegación de eventos** (poner un solo listener en el contenedor en vez de uno en cada botón): la tabla de eliminar se vuelve a dibujar después de cada cambio, y cada vez se crean botones nuevos. Si el listener estuviera en cada botón, se perdería al redibujar. El `<tbody>` nunca se destruye, así que su listener sigue funcionando. `evento.target` indica qué botón se tocó.
- **Después de cada cambio se redibuja todo** (`mostrarPlatos`, `mostrarEliminar`, `mostrarOpcionesEditar`). Es más simple que buscar qué parte del DOM (Document Object Model: la representación en memoria del HTML que JS puede modificar) cambió. Con 8 platos no se nota en el rendimiento.
- **Navegación con `:target` en CSS:** los enlaces del aside (`#listar`, `#crear`...) muestran solo el panel elegido, sin JavaScript.

5. localStorage: lo que más costó

El problema: cada página HTML corre con su propia memoria. Si creabas un plato en `gestion.html`:

- al recargar, desaparecía, porque las variables de JS se borran con cada recarga;
- `index.html` nunca lo veía, porque las páginas no comparten variables.

La solución: `localStorage` (almacenamiento del navegador que persiste entre recargas). Funciona **por origen** (mismo protocolo + dominio + puerto), no por página. Por eso `index.html` y `gestion.html` leen y escriben en el mismo lugar.

Tres detalles con su razón

1. **Solo guarda texto.** Por eso se usa `JSON.stringify` al guardar y `JSON.parse` al leer. Si guardas un array directamente, lo que queda guardado es el texto `"[object Object]"` y los datos se pierden.
2. **Se llama a `guardarPlatos()` después de cada `push`, `splice` o reemplazo.** Si falta en una operación, ese cambio vive solo hasta la próxima recarga.
3. **Por qué `length = 0` + `push(...)` en vez de reasignar:**

```
platosColombia.length = 0;
platosColombia.push(...JSON.parse(platosGuardados));
```

`platosColombia` es `const`, así que no se puede reasignar. Además, `script.js` y `gestion.js` ya apuntan a ese array. Vaciarlo y rellenarlo mantiene el mismo array con el contenido nuevo, y todos siguen viendo los datos actualizados. Es la diferencia entre **cambiar el contenido de una caja** y **cambiar de caja**.

6. Login con CREDENCIALES.JS

Cómo funciona: al cargar `gestion.html` solo se ve `#panel-login`. Si el usuario y la contraseña coinciden con los de `CREDENCIALES.JS`, se oculta el login y se muestra `#app-gestion` (`hidden = true/false`).

Lo que quedó claro: esto **no es seguridad real**. El archivo se descarga al navegador, así que cualquiera puede ver las credenciales en las DevTools, o saltarse el login escribiendo `appGestion.hidden = false` en la consola. Es un ejercicio de **mostrar u ocultar la interfaz según un estado**. La seguridad de verdad necesita un servidor (backend) que valide los datos sin mandárselos al navegador.

Dos cosas para tener en cuenta (no se corrigieron el viernes)

1. **Los platos que se crean en `gestion.html` salen sin foto en `index.html`.** La foto se busca con `imagenesConocidas[plato.nombre]`, y ese objeto solo conoce los 8 platos originales. Pasa lo mismo si editas el nombre de un plato existente: se pierde la conexión con su foto. **Concepto:** usar el nombre como clave une dos datos que pueden cambiar de forma independiente. Una solución sería agregar un campo `imagen` al formulario, o mostrar una imagen por defecto cuando no se encuentre la foto.
2. **Si guardaste datos en `localStorage`, los cambios que hagas en `data.js` ya no se ven.** Los datos guardados reemplazan al array original. Para probar desde cero: DevTools → Application → Local Storage → borra la clave `platosColombia`.

Conceptos que conviene repasar

- **Closures:** por qué cada tarjeta recuerda su plato.
- **Delegación de eventos:** por qué el listener va en el `<tbody>`.
- **Referencia vs. reasignación:** el truco de `length = 0` con `const`.